

# Expressions booléennes Structures de contrôle

# Sommaire

Expressions booléennes

Évaluation paresseuse

Structures de contrôle

# Expressions booléennes

$E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{not } E \mid ( E ) \mid \text{id relop id} \mid \text{true} \mid \text{false}$

Les expressions booléennes sont souvent employées dans la condition d'une structure de contrôle : **if**, **while**, **for**

## Évaluation stricte (*eager evaluation*)

On évalue toutes les sous-expressions

## Évaluation paresseuse (*short-circuit evaluation*)

On n'évalue une sous-expression que si c'est indispensable

**&&** et **||** en C, Java, C++

**and then** et **or else** en Ada

```
if (index<MAX &&  
    list[index]!=value )
```

# Évaluation stricte (*eager evaluation*)

## Code source

`b or x < y`

## Code intermédiaire

```
if x < y goto label1
tmp1 := $0
goto label2
label1 :
tmp1 := $1
label2 :
tmp2 := b or tmp1
```

Comme les expressions arithmétiques

0 représente faux, 1 représente vrai

Sauvegarder les valeurs des sous-expressions dans des variables temporaires

Si b vaut vrai, on exécute quand même la comparaison et le **ou** logique

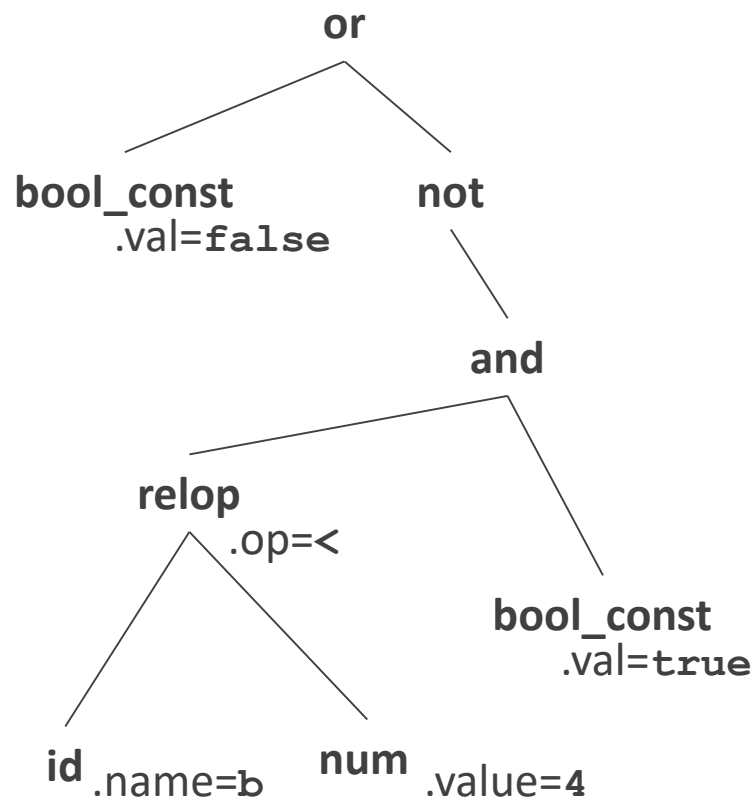
Sauts en avant

- On peut utiliser une étiquette symbolique sans savoir où sera l'instruction cible
- On crée l'étiquette d'abord, on écrit l'instruction cible après

**&** et **|** en C, Java, C++

**and** et **or** en Ada

# Évaluation stricte



Exercice : écrire en pseudocode un traducteur par parcours de l'arbre abstrait

# Traduire un arbre abstrait en code à trois adresses

Nœud **or**

```
visit(or.firstchild); visit(or.secondchild);
or.place = newtemp();
write(or.place ' := ' or.firstchild.place 'or' or.secondchild.place) ;
```

Nœud **and**

```
visit(and.firstchild); visit(and.secondchild);
and.place = newtemp();
write(and.place ' := ' and.firstchild.place 'and' and.secondchild.place) ;
```

Nœud **not**

```
visit(not.child);
not.place = newtemp() ;
write(not.place ' := not' not.child.place) ;
```

Nœud **relop**

```
visit(relop.firstchild); visit(relop.secondchild);
iftrue = newlabel(); after = newlabel(); relop.place = newtemp() ;
write(' if' relop.firstchild.place relop.op relop.secondchild.place 'goto' iftrue);
write(relop.place ' := $0 '); write(' goto' after) ;
write(iftrue ' : ' relop.place ' := $1 '); write (after ' : ' ) ;
```

```
if x < y goto label1
tmp1 := $0
goto label2
label1:
tmp1 := $1
label2:
```

# Sommaire

Expressions booléennes

Évaluation paresseuse

Structures de contrôle

# Évaluation paresseuse (*short-circuit evaluation*)

```
if (index<MAX &&  
    list[index]!=value )
```

## Avantages

Éviter une erreur dans la sous-expression court-circuitée

Rapidité si l'évaluation qu'on court-circuite est coûteuse : appel de fonction

## Ordre d'évaluation

Le résultat dépend de l'ordre dans lequel le programme évalue les sous-expressions

C, Java, C++ garantissent l'évaluation de gauche à droite pour les opérateurs booléens

# Évaluation paresseuse avec constantes booléennes

## Code source

b or x < y

## Code intermédiaire

```
if b ≠ $0 goto label1
```

```
if x < y goto label1
```

```
tmp2 := $0
```

```
goto label2
```

```
label1 :
```

```
tmp2 := $1
```

```
label2 :
```

Si b vaut vrai, on n'exécute pas l'autre  
comparaison

Si un nœud relop est fils d'un opérateur booléen,  
pas besoin de temporaire booléen pour le  
nœud relop

## Code source

$b \text{ or } x < y$

### Code intermédiaire

#### Évaluation stricte

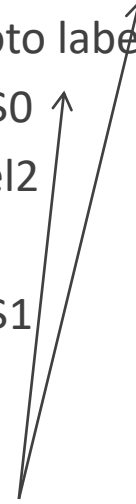
```
if x < y goto label1
tmp1 := $0
goto label2
label1 :
tmp1 := $1
label2 :
tmp2 := b or tmp1
```

## Comparaison

### Code intermédiaire

#### Évaluation paresseuse

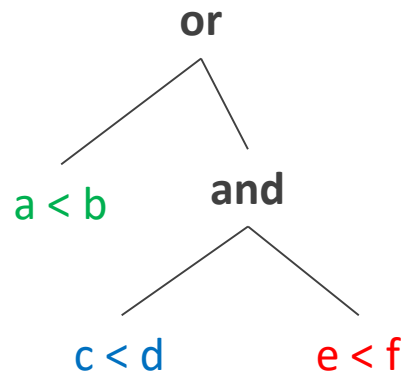
```
if b ≠ $0 goto label1
if x < y goto label1
tmp2 := $0
goto label2
label1 :
tmp2 := $1
label2 :
```



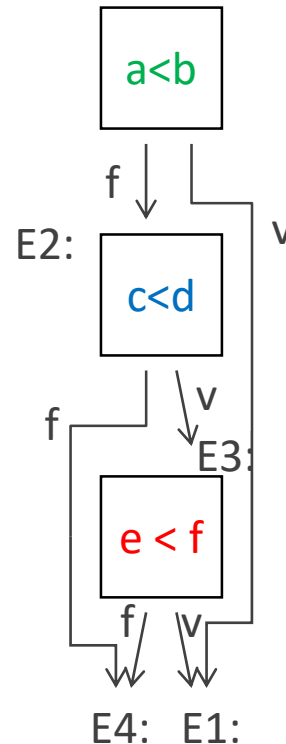
Plusieurs sauts vers la même étiquette

## Code source

$a < b$  or  $c < d$  and  $e < f$

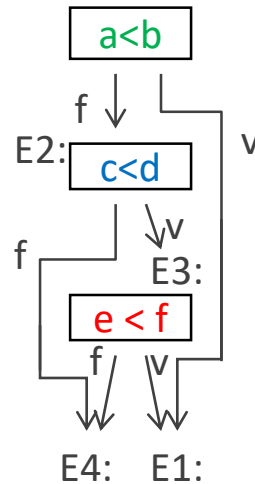


## Évaluation paresseuse par sauts



On ne sauvegarde aucune valeur booléenne avant la fin de l'expression complète

Exercice : traduire en code à trois adresses



## Évaluation paresseuse par sauts

### Code source

$a < b$  or  $c < d$  and  $e < f$

### Code intermédiaire

```

if a < b goto E1
goto E2
E2:
if c < d goto E3
goto E4
E3:
if e < f goto E1
goto E4
E1:
(...)
E4:
  
```

Plusieurs sauts vers la même étiquette

# Évaluation paresseuse par sauts

## Code source

$a < b$  or  $c < d$  and  $e < f$

## Code intermédiaire

if  $a < b$  goto E1

goto E2

E2:

if  $c < d$  goto E3

goto E4

E3:

if  $e < f$  goto E1

goto E4

E1:

(...)

E4:

Au début de la traduction d'une expression booléenne, on crée deux étiquettes

- une pour les cas où l'évaluation donne "vrai"
- une pour les cas où elle donne "faux"

ou, si c'est une sous-expression, on réutilise des étiquettes déjà créées

# Évaluation paresseuse par sauts

## Code source

$a < b$  or  $c < d$  and  $e < f$

## Code intermédiaire

if  $a < b$  goto E1

goto E2

E2:

if  $c < d$  goto E3

goto E4

E3:

if  $e < f$  goto E1

goto E4

E1:

(...)

E4:

Au début de la traduction d'une expression booléenne  $B$ , on crée deux étiquettes

- $B.true$  pour les cas où l'évaluation donne "vrai"
- $B.false$  pour les cas où elle donne "faux"

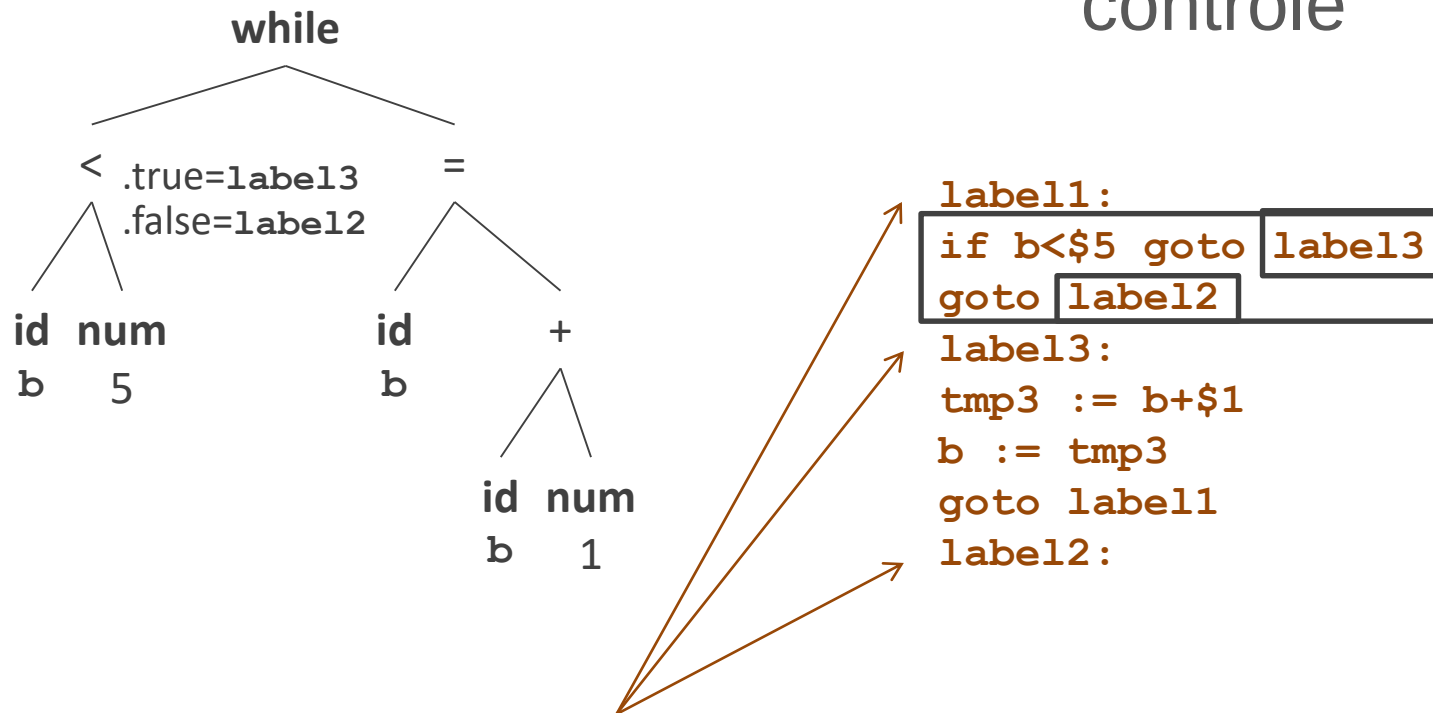
ou on réutilise des étiquettes déjà créées

## Sous-expression booléenne

$B.true$  et  $B.false$

- sont réutilisées
- ou sont créées par le nœud parent avant la traduction de la sous-expression

# Traduction des structures de contrôle

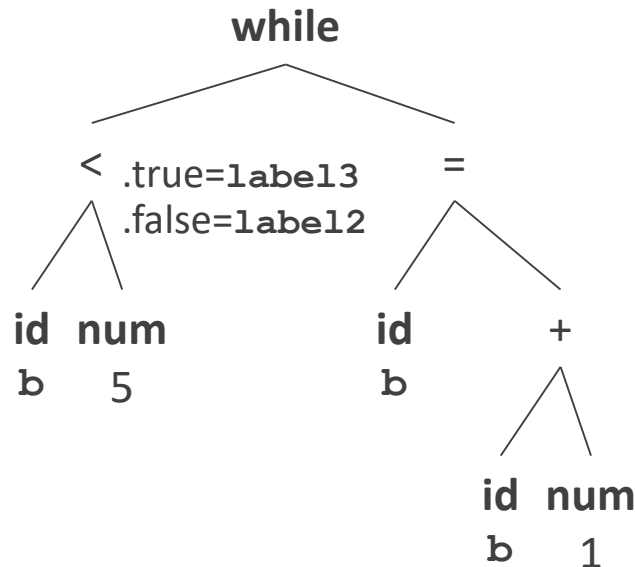


Pour traduire le **while**, on crée trois étiquettes

Pour traduire l'expression booléenne, on a besoin de connaître **label12** et **label13**

On peut les mettre comme attributs à la racine de l'expression booléenne

# Traduction des structures de contrôle



```

label1:
if b<$5 goto label3
goto label2
label3:
tmp3 := b+$1
b := tmp3
goto label1
label2:
  
```

Nœud  
**while**

```

begin = newlabel() ; after = newlabel() ;
iftrue = newlabel() ; write(begin ':' );
while.firstchild.true=iftrue; while.firstchild.false=after;
visit-sce(while.firstchild);
write(iftrue ':' ); visit(while.secondchild);
write('goto' begin); write(after ':' );
  
```

Une fonction de  
parcours différente  
pour passer en  
mode évaluation  
paresseuse

# Traduction des expressions booléennes

|                   |                                                                                                                                                                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nœud <b>Or</b>    | <pre>tmp = newlabel() ;<br/><b>Or</b>.firstchild.true = <b>Or</b>.true ; <b>Or</b>.firstchild.false = tmp ;<br/><b>Or</b>.secondchild.true = <b>Or</b>.true ; <b>Or</b>.secondchild.false = <b>Or</b>.false ;<br/>visit-sce(<b>Or</b>.firstchild); write(tmp ':') ; visit-sce(<b>Or</b>.secondchild);</pre>         |
| Nœud <b>And</b>   | <pre>tmp = newlabel() ;<br/><b>And</b>.firstchild.true = tmp ; <b>And</b>.firstchild.false = <b>And</b>.false ;<br/><b>And</b>.secondchild.true = <b>And</b>.true; <b>And</b>.secondchild.false = <b>And</b>.false;<br/>visit-sce(<b>And</b>.firstchild); write(tmp ':') ; visit-sce(<b>And</b>.secondchild);</pre> |
| Nœud <b>Not</b>   | <pre><b>Not</b>.child.false = <b>Not</b>.true ; <b>Not</b>.child.true = <b>Not</b>.false ;<br/>visit-sce(<b>Not</b>.child);</pre>                                                                                                                                                                                   |
| Nœud <b>relop</b> | <pre>visit(<b>relop</b>.firstchild); visit(<b>relop</b>.secondchild);<br/>write('if' <b>relop</b>.firstchild.place <b>relop</b>.op <b>relop</b>.secondchild.place 'goto'<br/>    <b>relop</b>.true);<br/>write('goto' <b>relop</b>.false) ;</pre>                                                                   |

# Affecter une expression booléenne à une variable

## Code source

```
found = (a < b);
```

## Code intermédiaire

```
if a < b goto E1  
goto E2  
E1:  
tmp := $1  
goto E3  
E2:  
tmp := $0  
E3:  
found =: tmp
```

L'évaluation paresseuse par sauts est commode pour les conditions des structures de contrôle  
**Si l'expression booléenne apparaît dans un autre contexte**

```
return a<b;
```

```
int my_condition = a<b ;
```

ajouter un temporaire pour sauvegarder la valeur de l'expression

**Si une expression numérique apparaît comme condition d'une structure de contrôle**

```
if (a) return a ;
```

ajouter une comparaison entre l'expression numérique et 0

# Exercice

Compléter le pseudocode du noeud "=" pour tenir compte de l'affectation d'une expression booléenne

Nœud

Pseudocode

=

```
visit_left(=.firstchild); visit(=.secondchild);
write( 'base [ ' =.firstchild.place ' ] :=' =.secondchild.place) ;
```

=

```
visit_left(=.firstchild);
```

```
write( 'base [ ' =.firstchild.place ' :=' =.secondchild.place) ;
```

# Sommaire

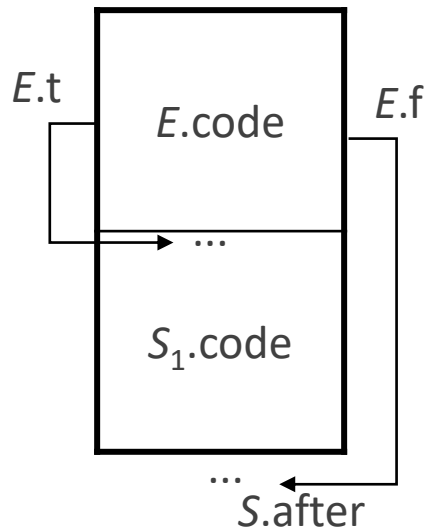
Expressions booléennes

Évaluation paresseuse

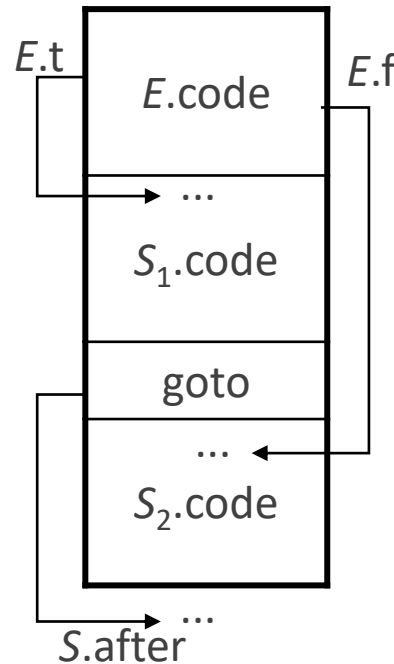
Structures de contrôle

# Structures de contrôle

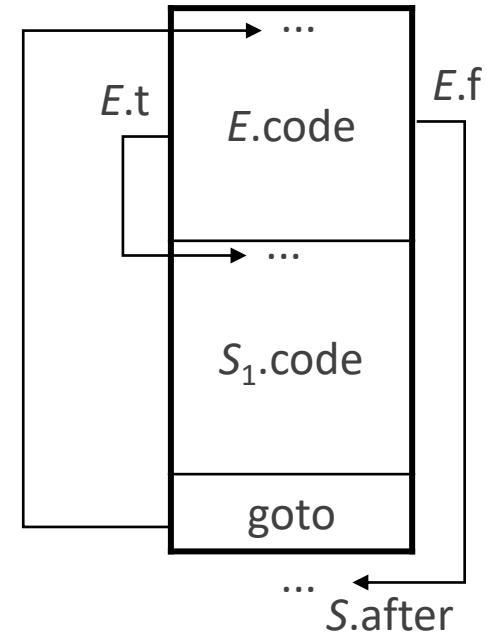
$S \rightarrow \text{if}(E) S \mid \text{if}(E) S \text{ else } S \mid \text{while}(E) S$



si-alors



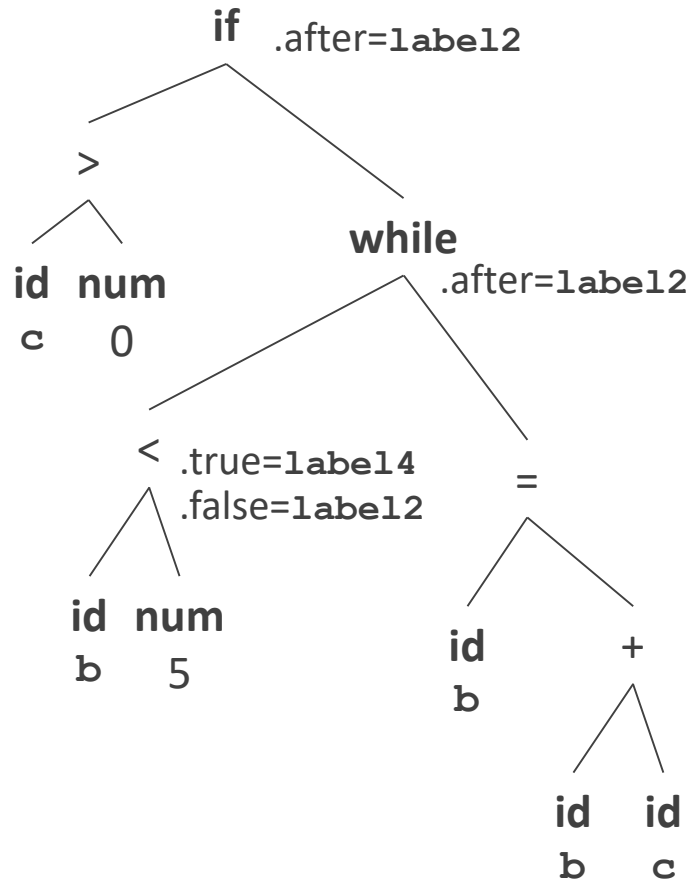
si-alors-sinon



tant que

On peut faire mieux en faisant converger plusieurs sauts

- vers la première instruction après la structure de contrôle
  - ou vers le début de la condition d'une boucle
- au lieu d'engendrer des chaînes de goto ou d'étiquettes



## Structures de contrôle

On peut récupérer l'étiquette `.after` à partir du nœud père au lieu d'en créer une à chaque structure de contrôle

# Évaluation paresseuse par sauts

Nœud **if**

```
iftrue = newlabel() ;  
if.firstchild.true = iftrue ; if.firstchild.false = if.after ;  
visit-sce(if.firstchild);  
if.secondchild.after=if.after;  
write(iftrue ':') ; visit(if.secondchild) ;
```

Nœud **while**

```
begin = newlabel() ; iftrue = newlabel() ;  
while.firstchild.true = iftrue ; while.firstchild.false = while.after ;  
write(begin ':') ; visit-sce(while.firstchild);  
while.secondchild.after = begin;  
write(iftrue ':') ; visit(while.secondchild) ; write('goto' begin) ;
```

# Évaluation paresseuse par sauts

Nœud **ifElse**

```
iftrue = newlabel() ; iffalse = newlabel() ;
ifElse.firstchild.true = iftrue ; ifElse.firstchild.false = iffalse ;
visit-sce(ifElse.firstchild);
ifElse.secondchild.after = ifElse.after ;
write(iftrue ':') ; visit(ifElse.secondchild) ; write('goto' ifElse.after) ;
ifElse.thirdchild.after=ifElse.after;
write(iffalse ':') ; visit(ifElse.thirdchild) ;
```

Nœud **L**

```
for each child S {
    S.after = newlabel() ; visit(S) ; write(S.after ':') ; }
write('goto' L.after) ;
```

# Merci

## CONTACT

ERIC LAPORTE

+33 1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR